



AI Mode ▾ All Images Videos News More ▾



how could I algorithmically determine a rhythm that will play well with a set of subdivisions of different ... ▾

October 26, 2025

To generate a rhythm that works with multiple cycles of different lengths (like 3 cycles of 3/4 and 2 cycles of 4/4), you must **find the total duration of the combined patterns**. This total duration, measured in the smallest common subdivision, is the length of your single, harmonizing rhythmic cycle. You can then use the Euclidean algorithm to evenly distribute a specific number of beats within that overarching cycle. [↗](#)

Step 1: Calculate the total length of the overarching rhythmic cycle

The first step is to establish the complete musical phrase that encompasses all the rhythmic cycles you want to combine.

1. **Find the beat count:** Determine the total number of beats for each of the rhythmic cycles.
 - 3 cycles of 3/4 time: $3 \text{ beats} \times 3 \text{ cycles} = 9 \text{ beats}$
 - 2 cycles of 4/4 time: $4 \text{ beats} \times 2 \text{ cycles} = 8 \text{ beats}$
2. **Find the least common multiple (LCM):** Since a quarter note represents one beat in both 3/4 and 4/4, the total duration in beats is the LCM of the total beat counts.
 - Find the LCM of 9 and 8. Since 9 and 8 are coprime (no common factors other

Ask anything





AI Mode ▾ All Images Videos News More ▾



$72 \text{ beats} \times 2 \text{ eighth notes per beat} = 144 \text{ subdivisions.}$ [↗](#)

This gives you a single rhythmic canvas 144 subdivisions long. Your $3/4$ cycle will repeat every $3 \times 2 = 6$ subdivisions, and your $4/4$ cycle will repeat every $4 \times 2 = 8$ subdivisions. Both will resolve together on the downbeat of the 144th subdivision. [↗](#)

Step 2: Use the Euclidean algorithm to generate a complementary rhythm

The Euclidean algorithm, also known as the Bjorklund algorithm, is a mathematical method for distributing a given number of "beats" as evenly as possible into a given number of "steps". This creates rhythms that are structurally balanced and widely found in world music. [↗](#)

To generate a rhythm that "plays well" with the underlying structure, you need two values:

- **Total steps (N):** The length of your overarching cycle in the smallest subdivision. For our example, $N = 144$.
- **Total beats (k):** The number of beat events you want in your new rhythm.

The number of beats (k) is a creative decision, but for good interplay, choose a number that is not a trivial factor of N (like 2, 4, or 8). Let's choose $k = 55$.

The algorithm works as follows (pseudo-code):

1. Initialize two lists:

- `beats` = a list of k beats, each represented as `[1]`
- `rests` = a list of $N - k$ rests, each represented as `[0]`



AI Mode ▾ All Images Videos News More ▾



↪ Provide a Python example to implement the Euclidean rhythm generation

↪ How can I adapt this approach to melodic or harmonic patterns?

↪ Tell me more about rotating the rhythm for variations



25 sites



Euclidean rhythm - Wikipedia

The Euclidean rhythm in music was discovered by Godfried Toussaint in 2004 and is described in a 2005 paper "The Euclidean Algorit...

Wikipedia

$$\begin{aligned} i &= q_0 k + r_0 = \\ i &= q_1 r_0 + r_1 = \\ i &= q_2 r_1 + r_2 = \\ &= q_3 r_2 + r_3 = \end{aligned}$$

Maths & Music: Euclidian Rhythms

Euclidean Rhythms resulted from an ancient mathematical concept - Euclid's Algorithm- being applied to music making in 2004 by a P...

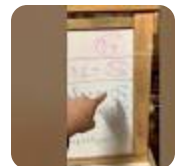
Top Music Arts



How to Count ANY polyrhythm (Rhythm Series Part 1) - YouTube

examples a poly rhythm is anytime rhythm interacts in a way where it can't be divided. into itself in other words there's two line...

YouTube



Show all



AI Mode ▾ All Images Videos News More ▾



are not coprime, the resulting rhythm breaks into multiple identical, evenly distributed sub-patterns. [↗](#)

Example: A rhythm with non-coprime beats and steps

Let's generate a rhythm with $k = 6$ beats distributed over $N = 16$ steps. In this case, the greatest common divisor (GCD) of 6 and 16 is 2. This means the Euclidean algorithm will produce a rhythm containing two identical sub-patterns.

The process to generate the pattern is as follows:

1. Set the parameters:

- **Total steps (N):** 16 (representing, for example, 16th-note subdivisions)
- **Total beats (k):** 6 (the number of events to distribute) [↗](#)

2. Visualize the algorithm:

- **Initial state:** Start with 6 groups of `[1]` and 10 groups of `[0]`.
 - `[1]` `[1]` `[1]` `[1]` `[1]` `[1]`
 - `[0]` `[0]` `[0]` `[0]` `[0]` `[0]` `[0]` `[0]` `[0]` `[0]`
- **Merge 1:** Take the first 6 `[0]` s and attach one to each of the 6 `[1]` s.
 - `[10]` `[10]` `[10]` `[10]` `[10]` `[10]`
 - `[0]` `[0]` `[0]` `[0]`
- **Merge 2:** There are now 6 groups of `[10]` and 4 groups of `[0]`. Take the 4 `[0]` s and attach one to the first 4 `[10]` s.
 - `[1001]` `[1001]` `[1001]` `[1001]`



AI Mode ▾ All Images Videos News More ▾



~ [10010] [10010] [100] [100]

o [10010] + [10010] + [100] + [100] = [10010100100] ⓘ

3. Resulting pattern: The final 16-step sequence is 1001010010010100 .

Notice that the pattern is simply the sequence 10010100 repeated twice. The length of this sub-pattern is $N/\text{GCD}(N, k) = 16/2 = 8$.

Implications for musical rhythm

- **Coprime numbers:** When k and N are coprime (e.g., $E(5, 16)$), the rhythm does not repeat until the full length of the cycle is complete. The rhythmic pattern feels more complex and syncopated relative to a simple pulse, creating a single, long phrase.
- **Non-coprime numbers:** When k and N are not coprime (e.g., $E(6, 16)$), the rhythm is more repetitive. The pattern can be heard as a smaller rhythmic motif that repeats an integer number of times within the larger cycle. The rhythmic "interest" comes from the interplay between this repeating sub-pattern and the primary pulse. ⓘ



↪ What are other ways to create rhythms from mathematical concepts?

↪ Let's explore non-coprime rhythms in more detail

↪ How does rhythm rotation affect the perceived groove?



11 sites





AI Mode ▾ All Images Videos News More ▾



medium

Circle of fifths and number theory - Applied Mathematics Consulting

We could play notes 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15. If m and n are not relatively prime, let d be their greatest common d...

 John D. Cook

Show all

what are strategies for employing rhythms that may start at different points in their cycles against a ...



October 26, 2025

Several effective strategies exist for using rhythms that begin at different points in their cycles against a single, unifying musical phrase. These techniques rely on manipulating the listener's perception of alignment and accent to create different effects, from subtle tension to dramatic phasing. [↗](#)

1. Phasing

Phasing is a technique, famously used by minimalist composer Steve Reich, where two or more identical rhythms start in unison but gradually move out of sync with each other. This is most often accomplished in one of two ways: [↗](#)

- **Gradual drift:** One rhythmic line slightly increases its tempo relative to the other. The rhythms slowly phase out of alignment, pass through a complex, intertwined section, and eventually realign perfectly. The process can repeat, creating an entire musical movement from a simple pattern.
- **Rotational shifting:** The rhythm of one voice is systematically rotated. For



AI Mode ▾ All Images Videos News More ▾



- And so on, until the two voices are back in sync.

2. Polymeter

Polymeter involves the simultaneous use of two or more time signatures or metric patterns against a common underlying pulse. The cycles are of different lengths and naturally shift their starting points relative to one another. [↗](#)

- **Establishing the supercycle:** As described previously, you can combine different meters (e.g., 3/4 and 4/4) by finding the least common multiple of their total beats.
- **Offsetting cycles:** Instead of starting all rhythms on the first beat of the supercycle, one or more can begin at a different point. For example, the 4/4 cycle could begin two beats into the larger phrase. All voices would still align at the conclusion of the supercycle, but the interplay between them would be different.
- **Metric modulation:** This technique can be used to smoothly transition from one meter or tempo to another. It uses a rhythmic pivot point—a subdivision that exists in both meters—to change the listener's perception of the beat. [↗](#)

3. Rhythmic canon

A rhythmic canon uses a single rhythm that is imitated by a second voice after a specific delay. This is similar to a melodic canon (like "Row, Row, Row Your Boat"), but it emphasizes the rhythmic element rather than the melody. [↗](#)

- **Simple canon:** A single rhythm is played, and a second instrument begins the same rhythm a few beats later. As the patterns overlap, new and complex-sounding rhythmic figures emerge. The "canon" resolves when the two voices



AI Mode ▾ All Images Videos News More ▾



- **Additive rhythms:** A basic rhythmic pattern is established, and a single subdivision is added to it with each repetition. The pattern grows and shifts its relationship to the underlying phrase over time.
- **Subtractive rhythms:** The reverse process occurs, where subdivisions are systematically removed from a pattern. [↗](#)

Example using subtraction:

- A phrase with eight subdivisions is established: 10101010 .
- The second repetition removes the last beat: 1010101 .
- The third repetition removes the second-to-last beat: 101010 .
- This creates a sense of rhythmic decay, where the pulse of the pattern gradually collapses against the single phrase. [↗](#)

5. Algorithmic variation

For a more automated approach, you can program algorithms to manipulate rhythm cycles in less predictable ways.

- **Randomized entry points:** In a multi-voice context, program each voice to begin its cycle at a random offset each time the phrase repeats.
- **Generative processes:** Use a generative algorithm (like the Euclidean algorithm) to create multiple rhythmic patterns. Then, design a process to modulate or randomly switch between these patterns throughout the musical phrase.
- **Chaos generators:** For more unpredictable results, you can use generative processes to introduce small, incremental, and slightly randomized shifts in timing to create an organic, non-mechanical phasing effect. [↗](#)



AI Mode ▾ All Images Videos News More ▾



↪ Tell me more about how metric modulation works



32 sites



Phase Shifting - Music Theory for the 21st-Century Classroom

Section 35.2 Phase Shifting. ... Your browser can't play this video. ... An error occurred. Try watching this video on www.youtube.com...



Music Theory for the 21st-Century Classroom



POLYMER - Understanding and Using Complex Rhythms ...

Poly meter involves the simultaneous use of two time signatures across different instruments or voices. For instance, a three to f...



YouTube



Polymeter / Wiki / Polymeter Techniques - SourceForge

Authors: Polymeter is the use of two or more meters simultaneously within a single piece of music. Meters are typically expressed ...



SourceForge



Show all

a pure data external that can apply quantization
and algorithmic variation and 0 to 100 percent ...



November 15, 2025

There is no single, dedicated external object in Pure Data (Pd) that precisely replicates the complex, specific feature set of the Soma Pulsar-23's quantizer and rhythm generator. However, you can build the desired functionality using existing



AI Mode ▾ All Images Videos News More ▾



- **[quant]** : This object is designed for pitch quantization but the core logic can be adapted for rhythmic timing or used as a reference for building a custom timing quantizer.
- **[tempo]** : This object helps manage the global tempo and provides timing information, which is essential for accurate quantization.
- **[cycle]** and **[periodic]** : These objects are useful for generating rhythmic events and managing the pulse, providing the underlying clock that your tapped-in rhythm will be quantized to.
- **[rand]** : This can be used to introduce the "algorithmic variation" aspect, generating random values within certain constraints to vary the recorded loop. [🔗](#)

Building a Custom Patch in Pure Data

You would need to combine several standard Pd objects and concepts to fully emulate the Pulsar-23's functionality for tapped-in rhythmic loops with variable quantization:

1. **Tap Tempo Input:** Use the **[timer]** and **[delwrite]** / **[delread]** objects to capture the precise timing of your tapped-in events in milliseconds and store them in a list or an array.
2. **Quantization Grid:** Use the **[tempo]** object or a combination of **[metro]** and **[counter]** to generate a master clock and a high-resolution timing grid (e.g., 64th notes).
3. **Variable Quantization (0-100%):** This is the core custom logic. You would compare the recorded tap times against the nearest "perfect" grid time.
 - **0% quantization:** The original, raw, tapped times are used.



AI Mode ▾ All Images Videos News More ▾



- Slight timing jitter (micro-timing).
- Probability-based skipping of certain hits.
- Algorithmic shifting of accents. [↗](#)

Resources for Learning and External Libraries

- [Pure Data Documentation and Community](#): The central hub for all things Pure Data, including links to downloads, forums, and community-made extensions. The forum is an excellent place to ask for help on specific patching challenges.
- **Pdida Library on GitHub**: Another library focusing on algorithmic and generative music that might offer useful abstractions or inspiration.
- [Pure Data Forum on real-time quantization](#): A forum thread where users discuss implementing real-time entry and quantization for drum machines, which addresses the core challenge of your request. [↗](#)



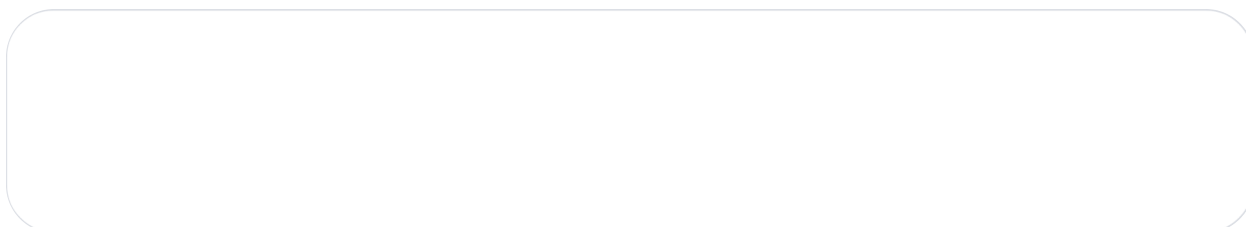
↪ Show me a patch example for rhythmic quantization in Pure Data

↪ Elaborate on how to implement variable quantization in Pure Data

↪ What are some examples of Pd abstractions for algorithmic variation?



56. Live Coding Toolkit for Pure Data - Part 2





AI Mode ▾ All Images Videos News More ▾



YouTube ⋮

56. Live Coding Toolkit for Pure Data - Part 2

The live coding toolkit is a collection of abstractions designed to enhance the efficiency of live coding in Pure Data by providin...

YouTube ⋮

Show all

would it be possible to build a c pure data external that can apply quantization and algorithmic ...



November 15, 2025

Yes, it is possible to build a custom C external for Pure Data (Pd) to handle quantization and algorithmic variation for tapped-in rhythmic loops. Building a custom external allows for optimized performance and a consolidated set of features tailored specifically to your needs, closely mirroring the functionality of the Soma Pulsar-23.

Why a C External is Suitable

- **Performance:** A C external can handle the intensive timing calculations and data manipulation more efficiently than a complex Pd patch made of many basic objects, which is crucial for real-time rhythmic processing.
- **Encapsulation:** All the complex logic for variable quantization and algorithmic variation is contained within a single object, simplifying your main patch.
- **Custom Features:** You can implement specific features not easily found in standard Pd objects, such as a dedicated 0-100% "swing" or "randomize timing" parameter that interpolates between raw input and a perfect grid.



AI Mode ▾ All Images Videos News More ▾



- **The Core Math:** The crucial part is the interpolation. For a given tapped time (T_{tap}), the nearest perfect grid time (T_{grid}), and a quantization percentage (P), the final output time (T_{out}) is calculated as:

$$T_{out} = T_{tap} + (T_{grid} - T_{tap}) \times P$$
- This formula smoothly transitions from the original tap time ($P = 0$) to the perfect grid time ($P = 1$).

4. Algorithmic Variation:

- **Jitter/Randomness:** Adding a small, user-controlled random offset to the final output time.
- **Probability Gates:** Adding the ability to randomly skip an event based on a probability setting.
- **Micro-rotation:** Shifting the entire recorded loop slightly forward or backward in time based on an input parameter.

5. **Playback Engine:** A mechanism within the C code to read the processed loop data and output bangs at the correct times in sync with the master clock.

Steps to Build the External

Building a C external for Pd requires some familiarity with the Pd API and C programming.

1. **Set up the Development Environment:** You will need a C compiler and the Pure Data source code headers or the Pd development kit for your operating system.
2. **Learn the Pd API:** Understand the fundamental functions like `class_new()`, `post()`, `outlet_new()`, `inlet_new()`, and how to handle messages (bangs,



AI Mode ▾ All Images Videos News More ▾



and effective way to achieve the precise rhythmic control you are looking for.



↳ Provide a basic C code structure for a Pd quantizer external

↳ What are some other generative music tools similar to Soma Pulsar-23?

↳ How does algorithmic variation work?

patterns can take a metric modulation transition in
and out. object will need to export and import ...



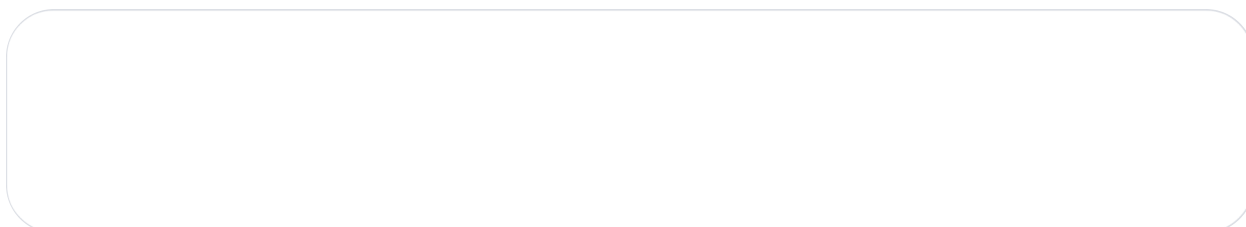
November 15, 2025

Building a C external for Pure Data (Pd) that handles sophisticated metric modulation and allows the import/export of pre-tapped patterns is achievable. This would extend the core functionality of your external beyond a simple quantizer to a full rhythmic manipulation tool.

Key Functionality Additions

The external would need to manage several complex aspects to support metric modulation and pattern saving/loading:

1. Metric Modulation Transition





AI Mode ▾ All Images Videos News More ▾



- **Tempo Ramping:** The object would calculate the required tempo change internally and use its master clock function to smoothly ramp the tempo up or down over a user-defined period (e.g., several beats or measures).
- **User Interface:** This would require additional inlets in your C external to receive commands like:
 - `[modulate_to_tempo X]`
 - `[modulate_to_meter Y Z]` (e.g., `[modulate_to_meter 6 8]`)
 - `[pivot_subdivision N]` (e.g., `[pivot_subdivision 8]` for an eighth note pivot)

2. Pattern Import and Export

The C external will need functions to communicate with the operating system's file system, which is a standard part of the Pd API. This allows you to save and load the internal data buffer that stores your tapped-in rhythms.

- **Exporting (Saving):**
 - The external would receive a `[save filename]` message.
 - It would then format the internal list of timestamps and other metadata (like the original tempo) into a plain text file (e.g., a simple space-separated list of numbers or a JSON file for more complex data).
 - The Pd API provides file handling functions (`fopen` , `fprintf`) that can be wrapped to operate safely within the Pd environment.
- **Importing (Loading):**
 - The external would receive a `[load filename]` message.
 - It would use the Pd API file handling functions (`fopen` , `fscanf`) to read the



AI Mode ▾ All Images Videos News More ▾



- **API Usage:** You'd rely heavily on specific Pd API functions for file access, which manage file paths relative to the current Pd patch and handle cross-platform compatibility.

This project significantly expands the scope but offers the power and flexibility to manage complex rhythmic transitions and a library of user-created patterns, creating a highly functional and versatile musical tool.



↳ What Pd API functions are useful for saving and loading patterns?

↳ Elaborate on how tempo ramping is implemented for metric modulation transitions

↳ Give an example of a custom data format for saving rhythms

algorithmic manipulation will be extensible allowing duplication of tidal cycles rhythmic features, like ...



November 15, 2025

Building a C external for Pure Data (Pd) that incorporates the rich algorithmic manipulation features found in systems like TidalCycles—such as generating palindromic rhythms—is a powerful extension. This functionality would reside within the "Algorithmic Variation" module of your C external.



AI Mode ▾ All Images Videos News More ▾



1. Palindromic Rhythms (Reverse and Combine)

A palindrome reads the same forwards and backward. In rhythm, this means playing a pattern and then its reverse immediately after.

- **Internal Function** `reverse_pattern()` : Your C external would need a function that reverses the order of elements in the internal data buffer.
- **User Command** `[palindrome]` : When this message is received, the external would:
 - Duplicate the current internal pattern data.
 - Reverse the duplicated data.
 - Append the reversed data to the end of the original data, creating a new, longer, palindromic loop.

Example:

- Original pattern: `1010` (4 steps)
- Reversed pattern: `0101`
- Palindromic result: `10100101` (8 steps)

2. Rhythmic Rotation

TidalCycles uses rotation (often called `rotate` or `shift`) to cyclically shift the start of a pattern. This is already a key part of algorithmic rhythm manipulation.

- **Internal Function** `rotate_by_n(int n)` : A C function to move the first `n` elements of the list to the end of the list.
- **User Command** `[rotate N]` : This inlet would provide immediate manipulation of the rhythmic feel without changing the underlying pattern data.



AI Mode ▾ All Images Videos News More ▾



- **Internal Logic:** The playback engine would need to manage two or more parallel rhythmic sequences, ensuring they all align correctly with the master clock, creating complex polymeters.

Extensibility of the C External

The beauty of building this in C is that you can design the internal architecture to be highly extensible.

- **Function Pointers:** You could use function pointers to dynamically switch between different manipulation algorithms, making it easy to add new TidalCycles-like features later (e.g., `[chop]`, `[euclid]`, `[every]`).
- **Data Abstraction:** By using a robust data structure (like a `struct` in C) to represent a "pattern," you make it easier to pass patterns to different manipulation functions and save them to disk.

By incorporating these features, your custom Pure Data external becomes a powerful, self-contained tool for both practical rhythmic quantization and advanced, algorithmic composition.



↳ Give a concrete example of applying palindrome transformation

↳ How can I visualize these complex rhythmic manipulations in Pure Data?

↳ How do you handle different time signature calculations for TidalCycles features?



AI Mode ▾ All Images Videos News More ▾



cleanup, and provides the basic inlets and outlets needed for a simple quantizer object, which we will call `quantizer_tilde` (following Pd naming conventions for audio/timing-related objects). [↗](#)

This structure includes:

- Standard Pd API includes and setup functions (`quantizer_tilde_setup`).
- A struct `t_quantizer_tilde` to hold the object's data.
- Basic methods (`quantizer_tilde_bang` and `quantizer_tilde_float` as placeholders for core logic).

C

```
#include "m_pd.h"
#include <math.h>
#include <string.h>

// Define the structure for our new object
static t_class *quantizer_tilde_class;

typedef struct _quantizer_tilde {
    t_object x_obj;
    // Outlet for sending out the quantized 'bang'
    t_outlet *x_outlet_quantized;
    // Outlet for sending out status messages/debug info
    t_outlet *x_outlet_status;

    // --- Object State Variables ---
    // Example variable: The current quantization percentage (0.0 to 1.0)
    t_float quantization_percentage;
    // Example: Storage for recorded tap times (this is where complex logic
    // In a real implementation, this would be a dynamic array or linked l
```



AI Mode ▾ All Images Videos News More ▾



```

    post("Quantizer received a bang! Percentage is %f", x->quantization_pe

// Send a 'bang' out the main outlet
outlet_bang(x->x_outlet_quantized);
}

// A generic float handler (e.g., for setting the quantization level)
void quantizer_tilde_float(t_quantizer_tilde *x, t_float f) {
    // Basic validation to keep the percentage between 0 and 100
    if (f >= 0.0 && f <= 100.0) {
        x->quantization_percentage = f / 100.0; // Store as a 0.0 to 1.0 v
        outlet_float(x->x_outlet_status, f); // Output the raw value for s
        post("Quantization set to %f%%", f);
    } else {
        post("Error: Quantization percentage must be between 0 and 100.");
    }
}

// The 'new' method (constructor)
// This is called when you create the object in a Pd patch (e.g., [quantiz
void *quantizer_tilde_new(t_floatarg f) {
    t_quantizer_tilde *x = (t_quantizer_tilde *)pd_new(quantizer_tilde_cla

// Create the main outlet (leftmost)
x->x_outlet_quantized = outlet_new(&x->x_obj, &s_bang);
// Create a secondary outlet for status messages (right)
x->x_outlet_status = outlet_new(&x->x_obj, &s_float);

// Initialize default values
x->quantization_percentage = 1.0; // Default to 100% quantization
x->last_tap_time = 0.0;

post("Quantizer external created.");

```



AI Mode ▾ All Images Videos News More ▾



```
quantizer_tilde_class = class_new(gensym("quantizer_tilde"),
    (t_newmethod)quantizer_tilde_new,
    (t_freemethod)quantizer_tilde_free,
    sizeof(t_quantizer_tilde),
    CLASS_DEFAULT,
    A_DEFFLOAT, // Accepts an optional creation argument (a float)
    0);

// Bind methods to messages
// The main inlet (leftmost) handles 'bang' messages
class_addbang(quantizer_tilde_class, quantizer_tilde_bang);
// The main inlet (leftmost) handles 'float' messages
class_addfloat(quantizer_tilde_class, quantizer_tilde_float);

// Add custom messages here as you build more complex features (e.g.,
// class_addmethod(quantizer_tilde_class, (t_method)quantizer_tilde_sa
// class_addmethod(quantizer_tilde_class, (t_method)quantizer_tilde_lo

post("Quantizer external finished setup.");
}
```

Use code with caution.



  2 sites



Enchanting Program Specification Synthesis by Large Language Models using Static Analysis and Program Verification

In detail, the algorithm works as follows: First, the algorithm initializes a code template `C` with the original code...



AI Mode ▾ All Images Videos News More ▾



pd external that accepts a list of midi notes as an argument message and the outputs each note ... ▾

November 15, 2025

This C code structure provides a basic Pure Data (Pd) external named `seq_notes`. This object accepts a list of MIDI note numbers via an `[argument]` message and outputs the next note in the sequence from its left outlet every time it receives a `[bang]` message. When the end of the list is reached, it loops back to the beginning.

C Code Structure for the `seq_notes` External

C

```
#include "m_pd.h"
#include <stdlib.h> // For malloc/free
#include <string.h> // For memcpy

// Define the structure for our new object
static t_class *seq_notes_class;

typedef struct _seq_notes {
    t_object x_obj;
    t_outlet *x_outlet_note; // Outlet for the MIDI note value
    int *notes;              // Pointer to store the list of notes
    int num_notes;           // The total number of notes in the list
    int current_index;       // The index of the next note to play
} t_seq_notes;

// --- Method Declarations ---
```



AI Mode ▾ All Images Videos News More ▾



```

        // Optional: Post a message if the list is empty
        // post("Sequence is empty, skipping bang.");
    }
}

// The 'argument' message handler: accepts a list of MIDI notes
void seq_notes_argument(t_seq_notes *x, t_symbol *s, int argc, t_atom *arg)
// Free any previously allocated memory
if (x->notes != NULL) {
    free(x->notes);
}

// Allocate memory for the new list of notes
x->notes = (int *)malloc(argc * sizeof(int));
if (x->notes == NULL) {
    // Handle allocation error
    x->num_notes = 0;
    post("Memory allocation failed for note list.");
    return;
}

// Copy the input arguments (atoms) into the notes array
for (int i = 0; i < argc; i++) {
    if (argv[i].a_type == A_FLOAT) {
        x->notes[i] = (int)atom_getfloat(&argv[i]);
    } else {
        // Handle non-float input if necessary, maybe set to a default
        x->notes[i] = 0;
        post("Warning: Non-float argument received at index %d.", i);
    }
}

x->num_notes = argc;

```



AI Mode ▾ All Images Videos News More ▾



```
// Initialize state variables
x->notes = NULL;
x->num_notes = 0;
x->current_index = 0;

return (void *)x;
}

// The 'free' method (destructor)
void seq_notes_free(t_seq_notes *x) {
    // Free dynamic memory when the object is deleted
    if (x->notes != NULL) {
        free(x->notes);
    }
}

// The setup function (called when the external is loaded by Pd)
void seq_notes_setup(void) {
    seq_notes_class = class_new(gensym("seq_notes"),
        (t_newmethod)seq_notes_new,
        (t_freemethod)seq_notes_free,
        sizeof(t_seq_notes),
        CLASS_DEFAULT,
        0); // No creation arguments for simplicity

    // Bind methods to messages
    class_addbang(seq_notes_class, seq_notes_bang);
    // Bind the 'argument' (or you can use a custom message name like 'lis
    class_addmethod(seq_notes_class, (t_method)seq_notes_argument, gensym(

    post("Seq_notes external ready.");
}
```



AI Mode ▾ All Images Videos News More ▾



3. **Send a list** using a message box connected to the left inlet:

- [argument 60 64 67 72]
- When this message is clicked, the external processes and stores the notes C4, E4, G4, and C5.

4. **Send bangs** to the left inlet (e.g., from a [metro 500]) to output the notes sequentially:

- The first bang outputs 60 .
- The second bang outputs 64 .
- The third bang outputs 67 .
- The fourth bang outputs 72 .
- The fifth bang outputs 60 again (looping).

5. **Connect the output** to a MIDI output object like [makenote] or a simple sound generator to hear the sequence.



↳ How can I adapt this C external for other data types?

↳ How to handle non-float input in seq_notes.c?

↳ Elaborate on using 'argument' vs. a custom message like 'list' for the notes in the Pd external



AI Mode ▾ All Images Videos News More ▾



a 4/4 bar). All rhythmic operations (like `fast`, `slow`, `stut`, or pattern stacking) are calculated as fractions or multiples of this abstract cycle length. [↗](#)

To implement this in a Pure Data (Pd) C external, you need a system that defines time relative to this flexible cycle rather than an absolute, fixed tempo.

Core Principles for the C External

1. **Time as a relative fraction of a cycle:** The internal data of your C external should store event timings not in milliseconds, but as **normalized float values between 0.0 and 1.0** (representing a full cycle).
2. **External tempo management:** A separate mechanism (either within the Pd patch using a `[tempo]` object from the Live Coding Toolkit, or another inlet in your external) should convert this abstract cycle duration into real-world time (milliseconds or samples). The `[tempo]` object is ideal as it manages the global beat and handles tempo changes cleanly.
3. **Algorithmic functions operate on normalized time:** All your TidalCycles-like algorithms (palindrome, rotation, etc.) should manipulate these 0.0 to 1.0 float values. This makes them inherently time-signature independent. [↗](#)

C External Implementation Strategy

Your `t_quantizer_tilde` struct would need to store timing data as normalized floats: [↗](#)

C

```
typedef struct _quantizer_tilde {
    // ... (previous variables) ...
    t_float *normalized_times; // Array of event times (0.0 to 1.0)
    int num_events;
```



AI Mode ▾ All Images Videos News More ▾



pattern's density, similar to how Tidal does it.

- `[cycle_len 4.0]` or `[cycle_len 3.0]` : An inlet for the user to define what a "full cycle" means in terms of quarter notes. This value would be sent to the external tempo management system (e.g., the `[tempo]` object). The C external only cares that it operates within the 0.0 to 1.0 range.
- `[fast N]` : An internal C function that processes the `normalized_times` array to speed up the pattern. This would involve calculating the new positions for each event, effectively remapping the 0.0 to 1.0 range `N` times within the same duration. The resulting pattern would loop `N` times as fast.
- `[slow N]` : The opposite of `fast`, spreading the pattern over a longer duration (e.g., `slow 2` makes the 0.0 to 1.0 cycle take twice as long). [↗](#)

Algorithmic Manipulation and Time

When implementing features like the `[palindrome]` command:

1. The algorithm reverses the order of the normalized times.
2. It also needs to calculate the new time values. If the original pattern's events were at `[0.0, 0.5]`, the reversed part needs to occur at `[0.5, 1.0]` (within the original single cycle duration).
3. The resulting list of normalized times `[0.0, 0.5, 0.5, 1.0]` is passed back to the playback engine. [↗](#)

By working exclusively with these relative, normalized timings, your C external remains highly flexible and can interact with any global tempo or perceived time signature the user sets in their Pd patch, precisely matching the philosophy of TidalCycles.

π ∂ ∫ —



AI Mode ▾ All Images Videos News More ▾



 Sundrax Electronics ⋮



IIGetRegionTimeDilation

Returns a float that is the current time dilation, the value range is [0.0, 1.0], 0.0 (full dilation) and 1.0 (no dilation).

 Second Life ⋮

Show all

